

Міністерство освіти і науки України
Національний університет “Полтавська політехніка імені Юрія
Кондратюка”
Навчально-науковий інститут інформаційних технологій та
робототехніки
Кафедра комп’ютерних та інформаційних технологій і систем

Дисципліна «Технології розробки корпоративних Web-додатків»
Лабораторна робота № 2

Виконав:
студент групи 401-ТН
Тимошенко Владлен
Олександрович
Перевірив: Тютюнник Петро
Богданович

Полтава 2026

1. Обґрунтування вибору технологічного стеку

Мова програмування Back-end:

Для реалізації серверної частини веб-додатку я обрав Node.js, оскільки він дозволяє використовувати JavaScript як на клієнтській, так і на серверній стороні, забезпечує високу продуктивність завдяки асинхронній моделі роботи та добре підходить для створення REST API.

Фреймворк Front-end:

Клієнтська частина буде реалізована за допомогою React, що дозволяє створити сучасний застосунок із швидким та динамічним інтерфейсом.

Система керування базами даних (СКБД):

Як основну базу даних я обрав MongoDB через її гнучку документо-орієнтовану структуру, яка зручно інтегрується з JavaScript.

Інструменти DevOps/Hosting:

Для контролю версій застосую GitHub, а для автоматизації тестування та розгортання — GitHub Actions, що забезпечить стабільність та ефективність процесу розробки.

2. Архітектуру моделювання за методологією C4

Рівень 1: System Context Diagram:

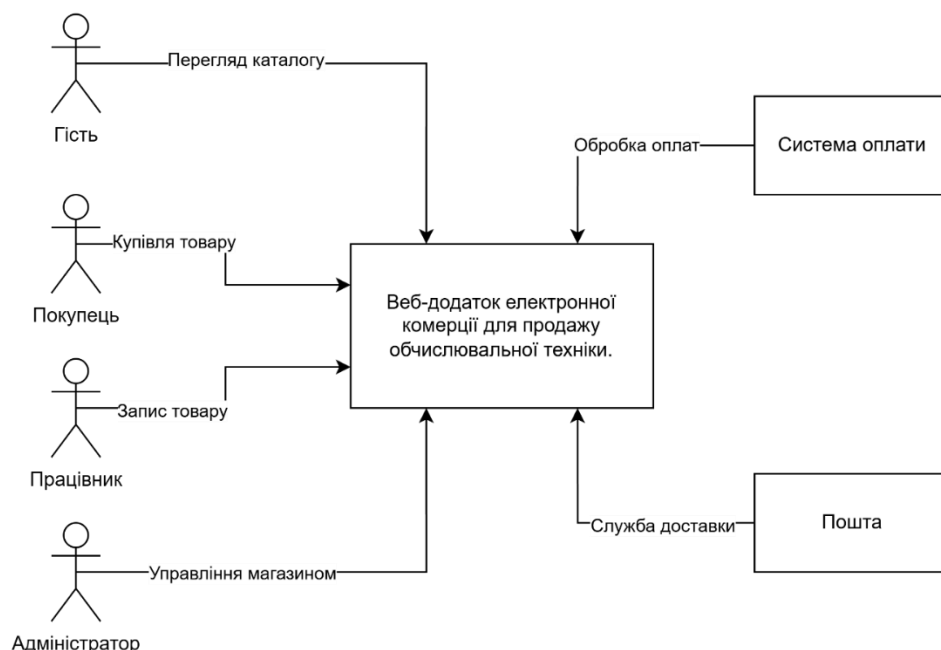


Рис 1 - System Context Diagram

Рівень 2: Container Diagram:

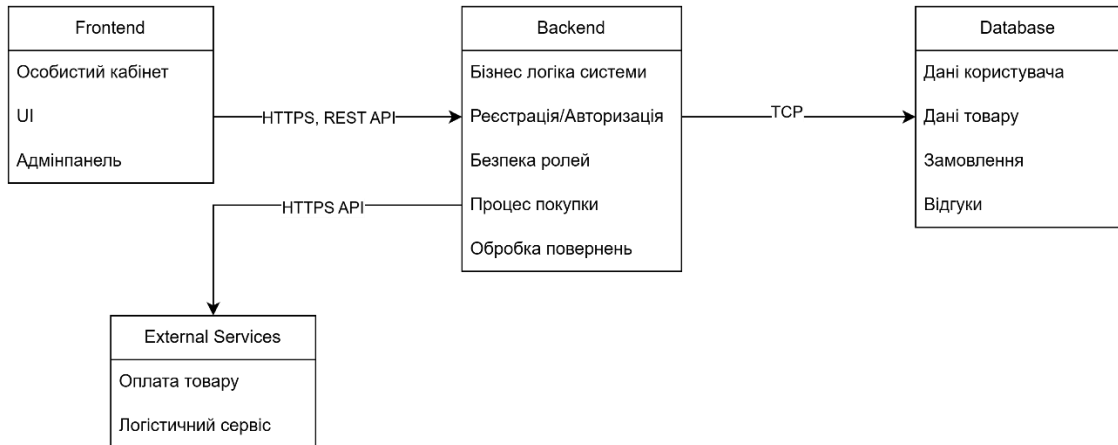


Рис 2 - Container Diagram

Рівень 3: Component Diagram:

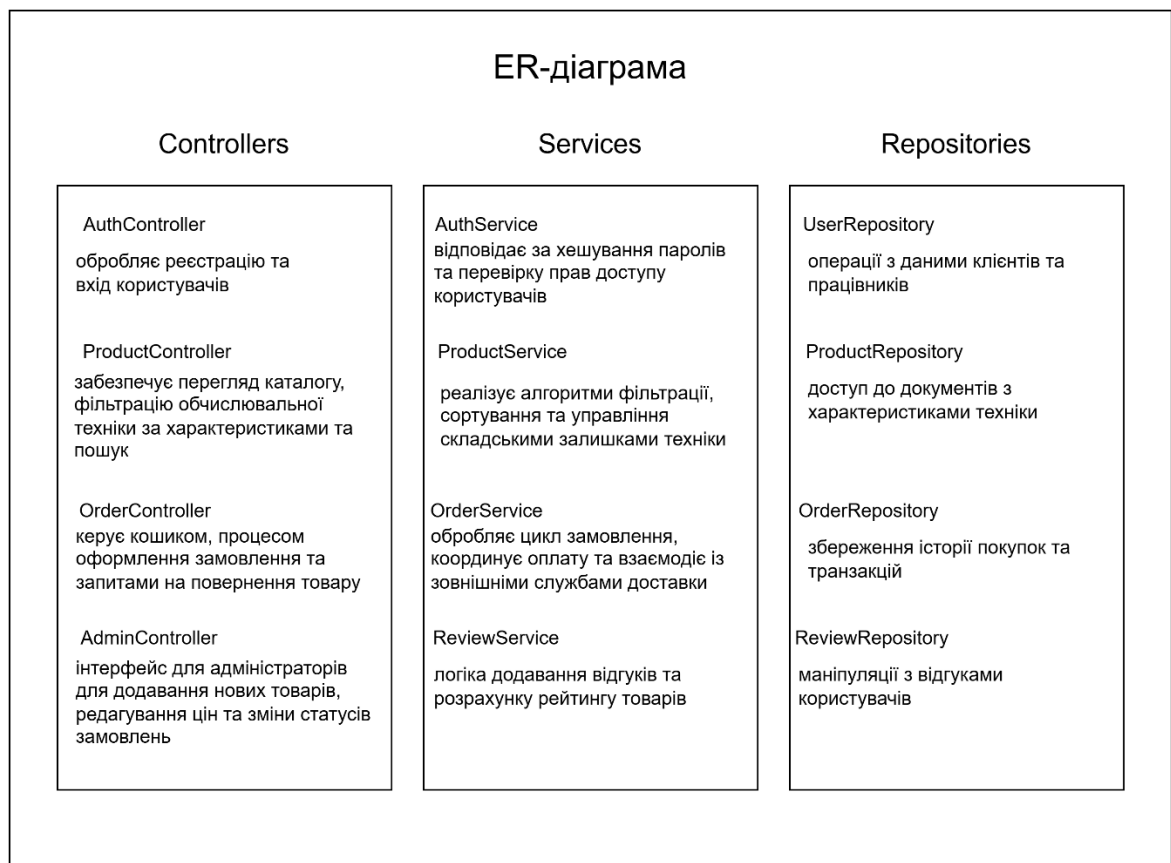


Рис 3 - Component Diagram

Контрольні запитання

1) Що таке модель C4 і для чого потрібен кожен з її рівнів (Context, Container, Component)?

C4 Model – це набір ієрархічних діаграм для візуалізації архітектури програмного забезпечення, розроблений Саймоном Брауном. Назва "C4" походить від чотирьох рівнів абстракції: Context (Контекст), Containers (Контейнери), Components (Компоненти) та Code (Код).

Основна ідея C4 Model – надати різні рівні деталізації, що дозволяють архітекторам розповідати "історію" архітектури системи різним аудиторіям, починаючи з високорівневого огляду і поступово занурюючись у деталі. Це як масштабування карти: від країни (Context) до будівлі (Code).

2) У чому принципова різниця між реляційними (SQL) та нереляційними (NoSQL) базами даних? Коли варто обирати кожен з них?

Принципова різниця між реляційними (SQL) та нереляційними (NoSQL) базами даних полягає у структурі даних (жорсткі таблиці з рядками/стовпцями проти гнучких форматів, таких як документи, графи, пари ключ-значення) та масштабуванні (вертикальне для SQL, горизонтальне для NoSQL). SQL забезпечує сувору відповідність транзакціям (ACID), тоді як NoSQL забезпечує високу швидкість та гнучкість при роботі з великими обсягами неструктурованих даних.

3) Що таке теорема CAP (Consistency, Availability, Partition tolerance) і як вона впливає на вибір бази даних?

CAP — теорема, яка стверджує, що при будь-якій реалізації бази даних неможливо одночасно забезпечити такі властивості: availability, consistency, partition tolerance. Теорема CAP змушує обирати між узгодженістю даних, доступністю системи та стійкістю до розділення, визначаючи вибір БД для розподілених систем. Розробники мають жертвувати або узгодженістю (AP), або доступністю (CP) під час збоїв мережі.

4) Які переваги та недоліки мікросервісної архітектури порівняно з монолітною (на рівні Container diagram)?

- Якщо ресурси обмежені й важливо мінімізувати витрати на інфраструктуру та DevOps, найчастіше стартують із моноліту. Якщо ж проєкт розрахований на довгу перспективу й закладається серйозний бюджет – має сенс одразу будувати мікросервіси.
- Коли потрібно швидко протестувати гіпотезу та запустити проєкт, моноліт дає вигоду у швидкості. Для продуктів, де важлива стійкість і незалежний розвиток модулів із перших днів, підійдуть мікросервіси.
- Якщо система розрахована на обмежений потік користувачів на старті – моноліту буде достатньо. Якщо очікується високе навантаження (наприклад, маркетплейс або фінтех-сервіс), мікросервіси виправдані з самого початку.
- Коли продукт планується як довгострокова екосистема, що включає різні напрями (CRM, склад, фінанси, мобільний застосунок), мікросервіси дозволяють будувати архітектуру, готову до розширення.
- Немає універсальної відповіді «моноліт кращий» чи «мікросервіси кращі». Архітектуру потрібно обирати під конкретний проєкт: моноліт – для швидкого старту, мікросервіси – для стійкого зростання.